DPU KV-cache Prefetch

黑化肥揮發會發灰｜國立成功大學｜NCHC Open Hackathon 2026 Final

OpenACC
More Science, Less Programming

Team ：黑化肥揮發會發灰

Team Members



王澤鈞
國立成功大學
電機工程學系



林宗賢
國立成功大學
電機工程學系



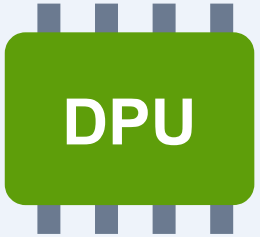
鄭智元
國立成功大學
電機工程學系

Mentor



Richard Huang
NVIDIA

DPU, KV cache, TTFT



DPU

BlueField-3

A small computer on the NIC that moves data into the GPU by itself.



KV cache

The model's notes

The notes a model keeps after reading the chat, so it never re-reads it.



TTFT

Time To First Token

The time from sending a question to the first word appearing.

The DPU moves the KV cache into the GPU early → shorter TTFT

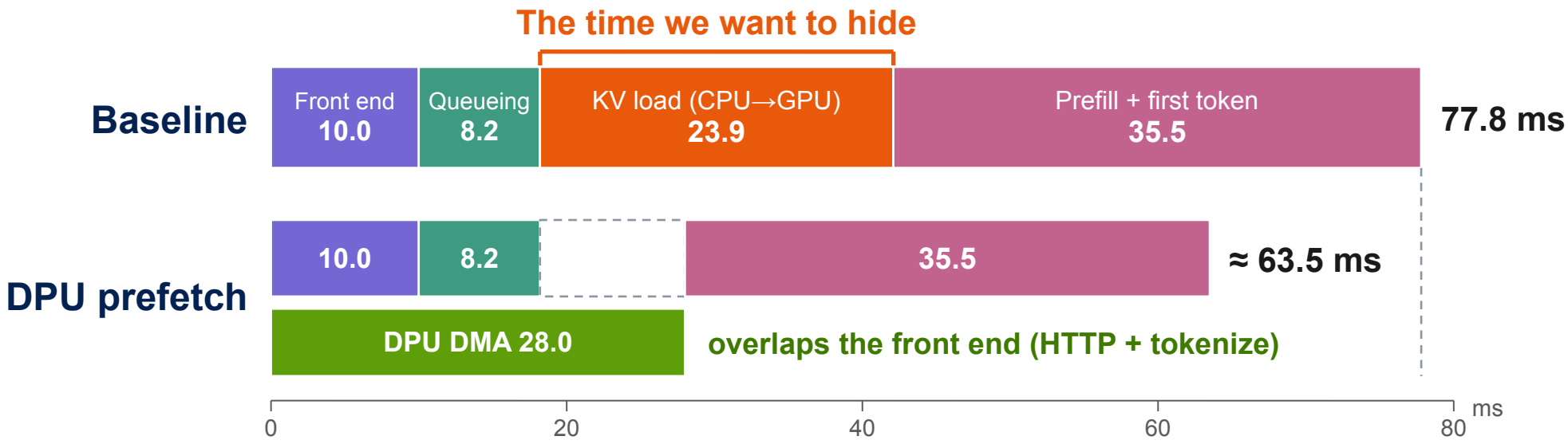
OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Application: the time we want to cut

Problem Every turn of a long chat moves 1.07 GiB of KV back into the GPU.

Method On arrival, the DPU moves the KV by DMA, overlapping the front end.

TTFT breakdown (warm, n=15, ms)

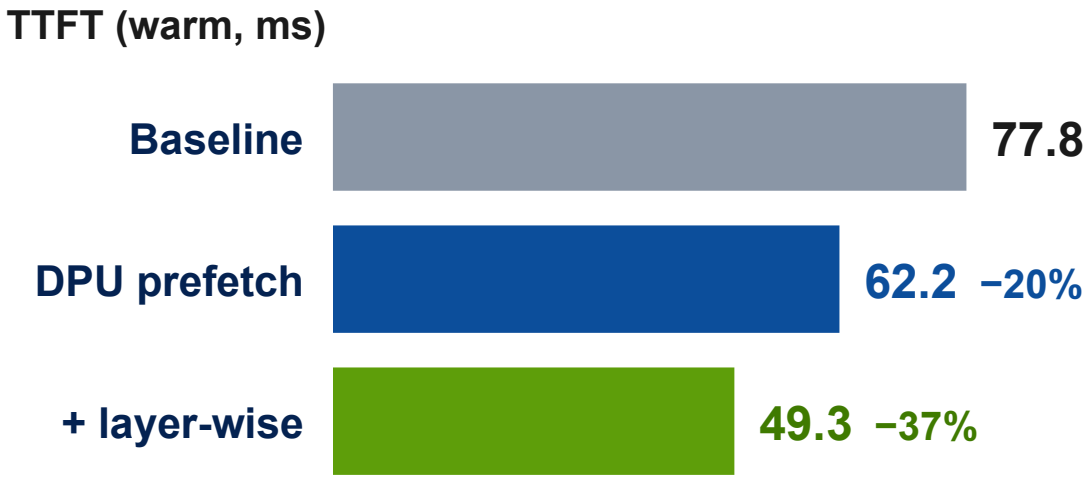


Qwen2.5-7B, vLLM 0.30, RTX PRO 6000, BlueField-3

Evolution and Strategy

Problem	Solution	Finding
vLLM allocates KV only after scheduling	GPU staging pool + in-GPU scatter	**KV copy overlaps the front end**
No public figure for DPU→GPU bandwidth	Measured it ourselves with DOCA DMA	**38 GiB/s: KV moved in 28 ms**
Driver conflict: DPU can't write to GPU	recompile peermem driver	**DPU writes directly into GPU**

Results: TTFT down 37%



−37%

≈ 1.6× faster

77.8 → 49.3 ms

Measured, n=15

host CPU full	DPU KV copy still 28.7 ms; saves 15–20 ms per request
4 concurrent	TTFT p50 drops from 121 to 51 ms
Correctness	189 wrong-guess tests, 0 wrong KV used

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Results: Multi-core vs GPU

Scenario: the same 20 requests, ~20k-token chat history.

	32-core CPU Threadripper PRO 9975WX	**GPU** RTX PRO 6000	**Speedup**
cold TTFT	66.05 s	1.15 s	57×
warm TTFT	2.89 s	77.9 ms	37×
decode	2.80 tok/s	91.9 tok/s	33×
Total time	**1214 s**	**32.6 s**	**37.3×**

37.3×

Total time 1214 s → 32.6 s

Energy Efficiency: 19× less energy

INPUTS	
# CPU Cores	32
# GPUs (A100)	1
Application Speedup	37.3x

Node Replacement	74.6x

GPU NODE POWER SAVINGS			
	AMD Dual Rome 7742	**8x A100 80GB SXM4**	**Power Savings**
Compute Power (W)	82,060	6,500	75,560
Networking Power (W)	3,464	93	3,371
Total Power (W)	**85,524**	**6,593**	**78,931**

Node Power efficiency	**13.0x**

ANNUAL ENERGY SAVINGS PER GPU NODE			
	AMD Dual Rome 7742	**8x A100 80GB SXM4**	**Power Savings**
Compute Power (kWh/year)	718,846	56,940	661,906
Networking Power (kWh/year)	30,346	814	29,533
Total Power (kWh/year)	**749,192**	**57,754**	**691,438**

$/kWh	$	0.18
Annual Cost Savings	$	124,458.88
3-year Cost Savings	$	373,376.65

Metric Tons of CO2	490
Gasoline Cars Driven for 1 year	106
Seedlings Trees grown for 10 years	8,104

(source: Link)

37.3× → 1 GPU node replaces 74.6 CPU nodes

Saves 78,931 W, ~490 t CO2 per year

※ Official calculator, standard-node estimate

19×

Same 20 requests

300.6 → 15.8 kJ

−97%

KV cache: per prefill

859 → 22.2 J

0.01

adaptive polling

Polling: 1 → 0.01 cores

Idle adds only 1.9 W

What problems have you encountered?

Existing code	vLLM has no arrival hook → hook added in the API server
Algorithm	Safe cancel of wrong guesses → sequence no. + hash check
Tool bug	NVML energy counter misreads at high rate → board power
Missing feature	DOCA 3.5 cannot export dmabuf memory
System config	DPU memlock limit of 3.9 GiB caps the KV store

Wishlist

Tools

- DOCA dmabuf memory export
- A request-arrival hook in vLLM

Language standards

None

Event

- More machine time
- A dedicated DPU + GPU node

Systems

- peermem auto-loaded at boot
- Adjustable DPU memlock limit

Final Thoughts

Was this Open Hackathon worth it?

Yes: a measurable DPU prefetch system, built from scratch.

Will you continue?

Yes

Next steps

- DPU writes directly into vLLM blocks
- Store KV on the DPU's NVMe
- Validate on real multi-tenant workloads

Support needed

- A dedicated BF3 + GPU test machine

Application Background

Every chat turn moves the KV cache back into the GPU.

The DPU starts the copy as soon as the request arrives.



TTFT 77.8 → 49.3 ms (−37%)

Measured, n=15

Hackathon Objectives and Approach

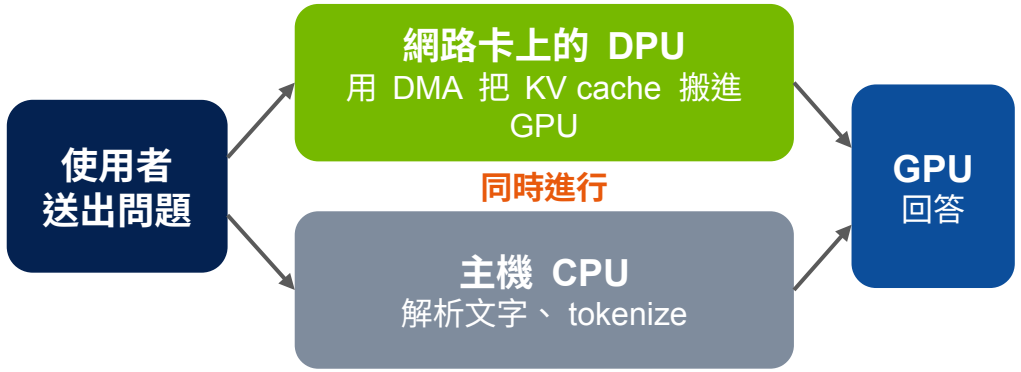
- Overlap the KV copy with the front end
- vLLM connector + DOCA DMA + CUDA

Technical Accomplishments and Impact

- TTFT −37%
- GPU speedup 37.3×, 19× less energy
- 189 wrong guesses, 0 wrong KV used

讓 AI 聊天機器人「記得」你的長對話，回得更快

示意：請求一到， DPU 搬 KV 與主機前處理同時進行



「黑化肥揮發會發灰」團隊來自蔡邦維老師帶領的 ITLab 實驗室， AI Agent 將 survey 、實驗與量測工作流程加速了約 10 倍； CPU/GPU/DPU 將長對話 LLM 推論加速了 37 倍！！

補充數據：多輪對話的首字延遲 warm TTFT （實測， n=15）



給一般讀者的說明

和 AI 聊天機器人長時間對話時，每問一句，系統都要重新讀一次累積約 2 萬個 token 的對話歷史，等第一個字出現的時間也跟著拉長。 KV cache 就像模型讀過歷史後留下的筆記，有了它就不必從頭重讀，但這份筆記有 1.07 GiB ，每一輪都得先搬進 GPU 。我們把筆記放在網路卡上的 DPU ，請求一進網卡， DPU 就自己把筆記搬進 GPU ，主機同時處理文字，兩件事一起進行。實測結果：第一個字的等待時間從 77.8 毫秒降到 49.3 毫秒，縮短 37% ；主機再忙， DPU 搬資料的時間也不受影響；就算 DPU 猜錯，也會自動重算，不會用錯資料。從 32 核 CPU 改用 GPU ，同一批工作快 37 倍、耗能省 19 倍。這能讓線上 AI 服務回得更快、更省電。


Thank You
OpenACC
More Science, Less Programming